A Report on the Implementation and Performance of a Image Processing Algorithm Using MPI

Exam No. B024703

December 3, 2016

1 Introduction

This report details the implementation and performance of a parallel message passing code. This code performs an iterative inverse edge detection algorithm using a 2d Von Neumann stencil. For this algorithm, the result of each output pixel depends on that of its neighbours in the input image. This made parallelising the code non-trivial, as it required boundary swaps to occur before each iteration. This was further complicated in this case by the selection of a two dimensional domain decomposition.

This report presents both a detailed description of the algorithm and its implementation, then shows the performance achieved with the resulting code, before concluding.

2 Methods

2.1 The Algorithm

The code is designed to perform the inverse of an edge detection algorithm. In that algorithm, each element, $edge_{i,j}$, is replaced as follows:

$$edge_{i,j} = image_{i-1,j} + image_{i+1,j} + image_{i,j-1} + image_{i,j+1} - 4image_{i,j}$$

To invert this algorithm and reconstruct the original image an iterative process must be used. In this process, for each iteration, an new two dimensional array is generated as follows:

$$new_{i,j} = 1/4(old_{i-1,j} + old_{i+1,j} + old_{i,j-1} + old_{i,j+1} - edge_{i,j})$$

After this the old and new arrays are swapped in memory and the process is repeated.

2.1.1 Boundary Conditions

The resulting value of $new_{i,j}$ in each case depends on its surrounding values in the *old* array. This presents an interesting problem as to what to do in the edge cases, where surrounding values do not completely exist.

In the vertical case, where the calculation required values that were above or below the existing *old* array, cyclical boundary conditions were used. This means that when the calculation required values one line below the *old* image, it instead used those from the top line of the *old* image and vice versa.

In the horizontal case, "sawtooth" boundary conditions were used, where values forming a smooth gradient ranging between 0 and 255 where used on both sides of the image. Detailed discussion of this boundary condition goes beyond the scope of this report.

2.1.2 Parallelisation

The code was parallelised using a two dimensional domain decomposition. This means that the initial 2d array containing our edge image was split into N_x sections in the x direction and N_y sections in the y direction, where $N_x \cdot N_y = P$, and P is the total number of processes.

For the same reasons as discussed in 2.1.1 the case of what to do at the edge of the image provides problems in parallelisation. To find the values required that are outside the range of each processes local array boundary swaps must occur. The neighbouring boundaries to each process's local image are appended to that image to form a halo, as shown in figure 1. This boundary swap was repeated before each iteration. Full details of how this was implemented are discussed in 2.2.

This means that once these halos are included, each process must perform the iterative process on an array of size $(N_x + 2) \cdot (N_y + 2)$.

2.1.3 Stopping Condition

For testing performance and accuracy compared to a serial version, it was useful to be able to terminate the algorithm after a fixed number of iterations. However, it is possible to calculate, according to a certain precision, when the iterative process is complete. This is done on the basis that once the algorithm has reached a perfect solution, if the algorithm is stable, then there will be no further changes between iterations. While this perfection may never be achievable, we can say that if the output changes below a certain amount then we can declare our image to be suitably similar to our original and terminate our iterative process.

In terms of calculation, this is done by calculating a value of $\Delta_{i,j}$ for every element in the two dimensional array:

$$\Delta_{i,j} = max(new_{i,j} - old_{i,j})$$

and comparing the largest of these values with a pre-determined value.

This stopping condition process is relatively straightforward to parallelise. Each process can calculate its own $max(\Delta_{i,j})$ and these individual values can be gathered and compared to find the maximum of these maximums by a master thread. Thinking in terms of Amdahl's law, it could be said that almost all of this stopping condition calculation is parallelisable, apart from a comparison of P floating point values, where P is the total number of processes.

Nevertheless, this process is relatively computationally expensive. To prevent this process having too strong a negative impact on performance, it was decided that it would only be completed every 20th iteration.

2.2 Implementation

The algorithm described above was implemented in parallel using MPI. However, message passing parts of the program are separated into functions. These message passing functions are contained in a separate source file. This allows the code to

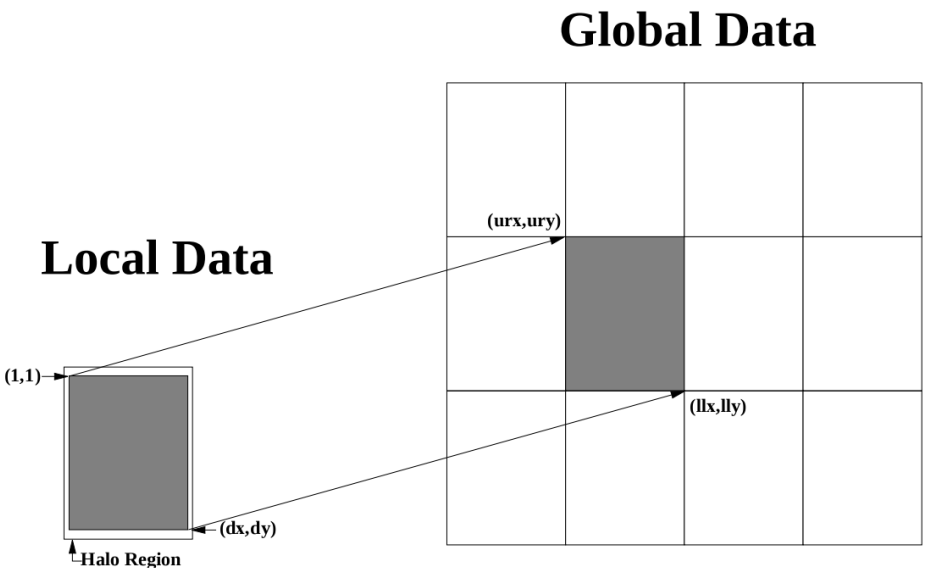

Figure 1: Diagram showing 2d decomposition and halo swap regions. Credit: [1].

be portable to another message passing framework should the need arise.

2.2.1 Initialisation and Cartesian Topology

While initialising message passing, a cartesian topology was implemented using `MPI_Cart_create`. This allowed the programmer to implement cyclic boundary conditions straightforwardly. The MPI implementation of a Cartesian topology includes a function for finding processes which are a certain displacement from a given process in a given direction. Cyclical boundary conditions in a specific direction can be requested with the initial call to `MPI_Cart_create`. Where this is done, if a process is requested that is outside the scope of the cartesian coordinate system, then the process at the other end of the system would be returned. In this case, cyclical boundary conditions were used in the vertical direction, meaning that when a process is requested that corresponds to being above the image, the process from the bottom of the image will be returned.

In directions where cyclical boundary conditions are not implemented and where a process is requested that is out of bounds, the `MPI_PROC_NULL` value is returned. Sends or receives from this null process will return immediately with no change to the receive buffer.

Both of these features of the MPI Cartesian topology make it easier to code more generally, ignoring the special cases at the edge of the image. Creating a Cartesian topology also allows the MPI implementation to arrange the processes in an optimal fashion for the hardware being used.

2.2.2 Scatter and Gather

The scatter and gather parts of the program form the beginning and end of the program. They are, however, discussed together here, due to their similarities.

Because a 2 dimensional domain decomposition was used, it was not easily possible to use the gather and scatter functions built into the MPI API.

Before the image could be scattered among the different processes, it had first to be read from a PGM file. This was done using the PGMIO program. This is done into a buffer, `masterbuf`, on the master rank. The master rank then loops through the other processes, writing parts of the `masterbuf` to a local image 2d array. Each array is then sent to its corresponding rank using `MPI_Send`. Those ranks which are not the master then receive this partial image, using `MPI_Recv`.

The gather process works much the same way but in reverse. First the master process writes its part of the image to the corresponding part of `masterbuf`. Those ranks which are not master then call `MPI_Send` to send their local arrays to the master, before the master process loops through each other process calling `MPI_Recv` to each, and writing each of these to the correct part of `masterbuf`. The master process then writes this masterbuf to a PGM file using the PGMIO program.

As a test, before the main part of the algorithm runs, just after the program calls the scatter function, the gather function is called, writing the input edge file out. This means that the correctness of these scatter and gather functions can be verified. Using the GNU `diff` tool, the file that was written out by the gather process was compared to the original input file and found to be the same, confirming that these scatter and gather functions work correctly.

2.2.3 Halo Boundary Swaps

As shown in figure 1, each decomposed local image must have a halo region containing a row or column from each neighbouring process. Before each iteration mes-

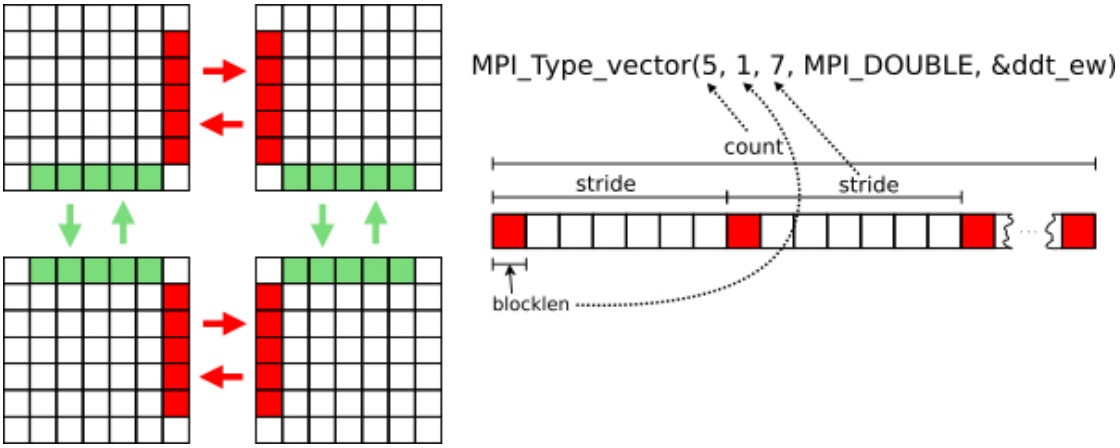

Figure 2: Derived data type for sending subvectors not contiguous in memory. Credit: [2]

sages must be passed to and received from each neighbour. As the halo boundary swaps must occur every iteration, this was very important to optimise for performance.

In C only 2d array elements in the vertical direction are contiguous. This means that a derived data type had to be used to send non-contiguous blocks of memory. To do this the `MPI_Type_vector` functions was used, which allows the programmer to specify which elements are selected from an array. How this applies to a 2d array is shown clearly in figure 2.

To exchange this data non-blocking communications were used, specifically the `MPI_Issend` and `MPI_Irecv` functions. This meant that the programmer had the option to perform some calculations while waiting for the message to be sent and received. As the majority of calculations in this image processing application do not depend on the halo region, this could prevent performance degradation on systems with high message passing latency. This effect is therefore more likely to be appreciable on instances where more than one node is used. The `MPI_Wait` function allows the programmer to synchronise processes, ensuring that all swaps are complete before halo regions are used.

2.2.4 Max Delta

The max Δ value discussed in 2.1.3 was implemented by first calculating a local delta value for each process, then finding the maximum of these values on all

processes using `MPI_Allreduce`. This function allows the programmer to easily find the maximum value across all processes and share this with all processes.

2.3 Compiling and Running

The code was compiled and run using the PGI MPT implementation of MPI on Cirrus, an Intel Xeon powered cluster. The `-O3` compiler flag was used to ensure maximum serial compiler optimisation.

3 Results

3.1 Correctness

As discussed in 2.2.2, the GNU `diff` tool was used to verify that the scatter and gather functions worked correctly. This same tool was also used to verify the end result. The source code that was provided to the author to parallelise was compiled in its original form, and outputs generated for each image used in the following parallel experiments. This output was then compared to all results yielded from the parallel versions of the code.

Unfortunately it was not possible to generate an accurate image using the MPI program discussed in this report. As shown in figure 3, there are clear artefacts in the generated image.

3.2 Scalability

Unfortunately, as well as yielding incorrect results, the code was not stable enough to perform a solid examination of its performance. The program would not reliably complete, often dead locking, and would not run at all when the domain was not decomposed by the same extent in both directions.

Had it been possible to do so it would have been interesting to compare how the speed up changed with the image size. As the message size is proportional to the perimeter of the image and the calculation size is proportional to the square of the side of the image, it seems likely that parallel efficiency will increase as the image

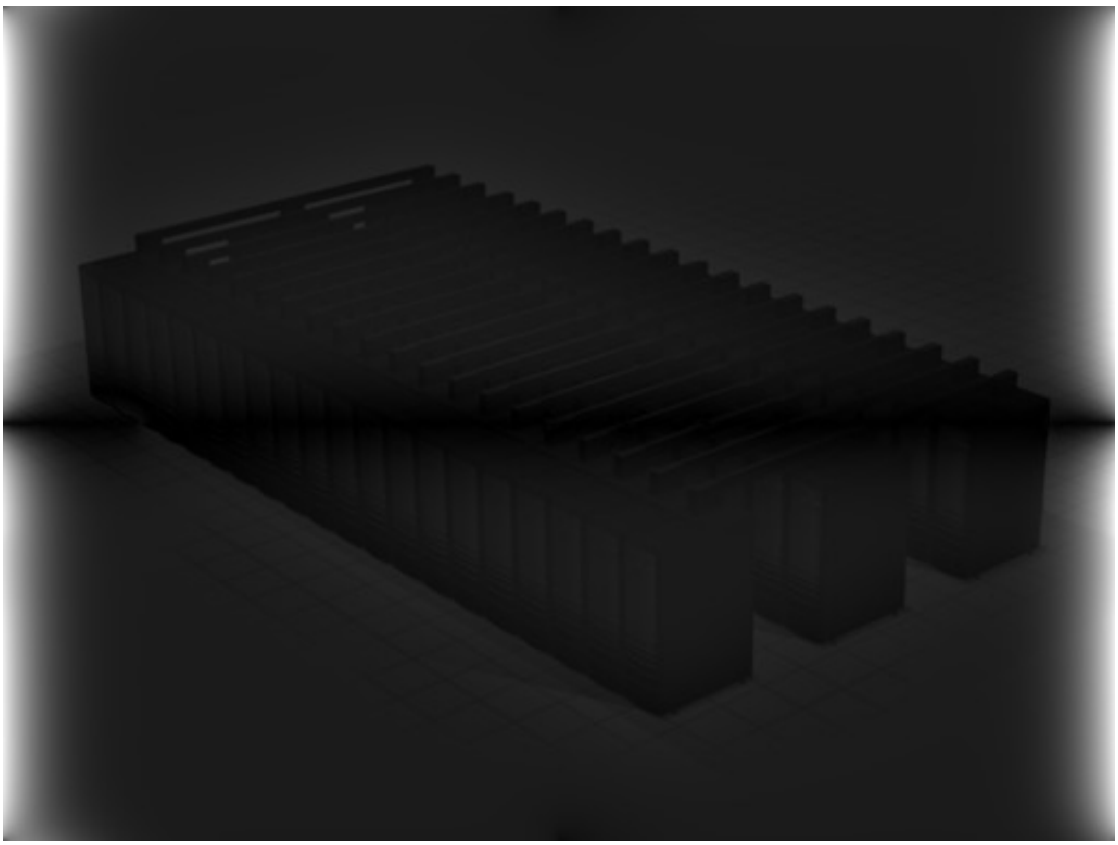

Figure 3: Image generated from 2x2 domain decomposition. It clearly shows artifacts from Halo swapping or boundary conditions.

size becomes larger. This is because, to achieve maximum parallel efficiency, it is desirable to have a high ratio of compute time to message passing time. More formally, it could be said that the halo swaps are $O(N)$ whereas the calculation itself is $O(N^2)$. This means the calculation becomes dominant where N is large.

It also would have been of interest to implement the technique described in 2.2.3, where elements that do not depend on the halos are calculated first. It seems unlikely that this technique would yield performance improvements when the latency of message passing is low. Indeed, it could have a negative effect due to losing the cache efficiency of performing all calculations together. A further experiment would be to see the effect of this technique when several nodes are used. In this case this technique is more likely to be effective.

The code was programmed such that it would be trivial to switch from using single precision floating point numbers to double precision floating point numbers. Experimenting with how doing this would affect performance would also have been worthwhile.

4 conclusion

To conclude, it was deeply disappointing that a working program was not produced. This made it impossible to verify the hypotheses proposed in 3.2.

The MPI code was found very difficult and time consuming to debug, which as well as making the end result unusable, prevented the author from exploring other more interesting performance aspects of the code.

However, the code is at least well laid out and provides a good basis for another programmer to start from when tackling this problem. Conducting this process also gave the programmer some insight as to the message passing paradigm and its idiomatic techniques, such as halo swapping, the creation of topologies and reductions. This is hopefully demonstrated to some extent in this report.

References

[1] Neil MacDonald, Elspeth Minty, Joel Malard, Tim Harding, Simon Brown, Mario Antonioletti, *Writing Message Pass-*

ing Parallel Programs with MPI, https://www.learn.ed.ac.uk/bbcswebdav/pid-2077851-dt-content-rid-3833626_1/courses/PGPH110782016-7SS1SEM1/MPP-notes.pdf Accessed: 26-10-2016

[2] SPCL *libpack: Runtime compiled pack functions* https://spcl.inf.ethz.ch/Research/Parallel_Programming/MPI_Datatypes/libpack/ Accessed: 26-10-2016